# Harness Engineering：11 个编码 Agent 的源码解剖·小白指南

原帖：@程序员鱼皮「有篇论文拆了 Claude Code 等 11 个编程 Agent 的源码，没有一个靠 LangChain」（291 赞）。

学习对象：arXiv:2609.00006（CC BY 4.0，Wavestone AI Lab，全文已存本机；**宣称核对 7/7 真实通过**——每条原句级命中）。

知识学习类 | harness 工程 | 源码解剖 | 七子系统 | 原句级核对

## 📖 疑问目录


Q1 这是什么 | Q2 什么是 harness | Q3 七子系统 | Q4 双缺席

Q5 最小与最大 | Q6 扩展标准之争 | Q7 平台转向 | Q8 与 vault 关系

Q9 给我什么能力 | Q10 动手实验

## Q1 这篇论文是什么？




一句话：**harness 工程这个 2026 年初被命名的学科，拿到的第一份系统性实证基础**——对 11 个生产级编码 Agent（Claude Code、Codex CLI、Gemini CLI、Mistral Vibe、OpenHands、Aider、Mini-SWE-Agent、Hermes、Pi、OpenCode、OpenClaw）加 Omnigent 元对照做源码解剖。方法论纪律极严：不 benchmark、不排名，只描述与比较。

| 事实项 | 核实值（原句级核对 7/7） |
| --- | --- |
| 论文 | arXiv:2609.00006v1（2026-07-15），CC BY 4.0，Wavestone AI Lab（Barbaste 等） |
| 语料规模 | 约 400 万行 Python/TypeScript/Rust（三次语料扩充后双缺席仍成立） |
| 产出 | 13 条横切观察 + 29 个反复出现的设计模式目录 + 五轴权衡框架 |
| 本机验证 | paper.html 全文存档 + verify_claims.py 原句级断言 7/7 |

## Q2 什么是 harness？什么不是？




论文定义：**Agent = 模型 + harness**——harness 是把 LLM 耦合到世界的运行时：循环、工具、上下文管理、安全控制、编排、扩展面。这个学科 2026 年初被命名（Anthropic 的 effective-agents 系列是其实证前奏）。关键边界：harness 不 benchmark 也不排名——它回答"怎么建的"，不回答"哪个更强"。

## Q3 七个子系统——所有 Agent 的共同解剖结构




七子系统（每个都有"最小实现→最大实现"的光谱，Table 1 全文映射）：

| 子系统 | 最小形式 | 最大形式 |
| --- | --- | --- |
| Agent loop | Mini-SWE-Agent：单个 bash 工具上的线性 while | OpenHands：持久事件日志上的事件溯源对话+并行动作批 |
| LLM 集成 | Mini-SWE-Agent：一次 LiteLLM 调用+一个 Jinja 模板 | Hermes：5 条自有传输+29 供应商档；Codex：服务端下发模型目录 |
| 工具与动作 | Mini-SWE-Agent：只有 bash | Claude Code：43 个类型化工具延迟加载；Codex：工具调用即 V8 代码 |
| 记忆与上下文 | Mini-SWE-Agent：无界线性历史 | Codex：agent 自维护跨会话记忆管线；Gemini CLI：图式上下文蒸馏 |
| 安全与权限 | Mini-SWE-Agent：成本+步数两限 | Codex：策略规则+LLM 审批员+三平台 OS 沙箱（四层栈） |
| 编排 | Aider：无（单 agent 设计） | Claude Code：递归组合；Omnigent：跨厂商协调 |
| 扩展 | Mini-SWE-Agent：结构化类型（Python 协议） | Pi：一切皆扩展运行时；Codex：市场分发插件 |

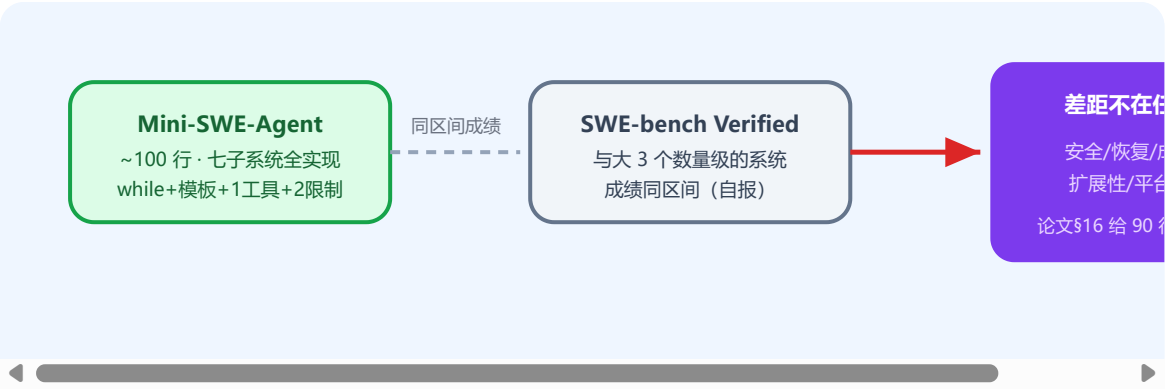
## Q4 "双缺席"——论文最炸的两个发现




① **无 agentic 框架**：约 400 万行代码里，没有任何 agent runtime 导入 LangChain/LangGraph/AutoGen 等通用框架——**连 Gemini CLI 都不用 Google 自家的**。全靠手写 async 循环。② **无向量代码检索**：没有一个系统用向量嵌入检索代码——全部是确定性检索：ripgrep、tree-sitter、glob、自动发现的 Markdown 上下文文件。两个缺席在三次语料扩充后依然成立。

**为什么重要：这与"RAG 一切"的主流叙事正面相撞。生产级 Agent 的答案是：代码检索用确定性工具（grep 系）因为精确；agent 骨架手写因为框架的抽象不适合循环的本质。**

## Q5 地板很低：100 行的 Mini-SWE-Agent




存在证明：Mini-SWE-Agent 用约 **100 行**实现全部七子系统——一个 while 循环、一个模板、一个工具、一个消息列表、两个限制、无编排、结构化类型作为全部扩展故事——SWE-bench Verified 成绩与体积大三个数量级的系统同区间。**地板与生产系统的差距不在任务完成，而在安全、恢复、成本管理、扩展性、平台面**。论文第 16 节甚至给出 90 行最小可行 harness 供复制特化。

## Q6 扩展标准之争：SKILL.md 赢了 MCP?




采用率：SKILL.md 技能 **9/11** 领先 MCP **8/11**；ACP 协议进入 6 个系统并获得第三个角色——harness 托管（OpenHands 把 Claude Code/Codex/Gemini CLI 当可互换后端跑）。技能生态长出了注册表、信任分级、以及语料中第一批 agent 写给 agent 的技能。论文判断："Skills versus MCP: the tie has broken"——平局已破，技能是声明式程序，延迟加载是主导模式。

## Q7 平台转向：CLI 正在变成框架




论文第 14 节的论题：harness 与框架正在合并。四个信号：① 技能成为声明式程序；② hooks/事件总线成为扩展基底；③ 工具与工作流的边界消失；④ harness 成为服务面（ACP 托管）。配套经济学：市场、切换成本、企业治理。这意味着**"用哪个框架"的问题正在被"用哪个 harness"吞掉**——因为 harness 自己长成了框架。

## Q8 与 vault 已有知识的关系

直接升级：[[AgentHarness智能体挽具]] 概念从"问题在 harness 不在模型"的单点认知，获得七子系统解剖+五轴权衡的体系化框架；[[工具目录膨胀]] 对应论文 8.3 延迟工具加载（Claude Code 43 工具是正面案例）；[[Compaction上下文压缩算法]] 对应 9.1-9.5 策略分类学（七系统阈值压缩）；[[工具调用三段式流水线]]、[[上下文工程四操作]] 均有论文级背书。vault 的 harness 品味区被这篇论文整体"正名"。

## Q9 这个项目给我什么（能力清单）

① **七子系统解剖眼**：看任何 Agent 先找七个位置，缺位也是位置。

② **最小/最大光谱思维**：每部件都有极简与极繁解，选型=在光谱上定位。

③ **双缺席判断力**：框架与向量检索的缺席是实证事实，不是品味。

④ **安全即架构**：Codex 四层栈证明安全不是提示词是运行时层。

⑤ **平台化嗅觉**：harness 吞框架，扩展面=生态入口。

⑥ **方法论纪律**：不 benchmark 只解剖——描述性研究的价值样本。



## Q10 动手实验：宣称原句级核对 7/7




实验文件：`exercise/verify_claims.py`（纯标准库，对已存档全文 paper.html 做原句级断言）。真实运行（exit=0）：

```
核对结果：7/7 通过
  ✅ T1 十一系统 11/11 + Omnigent=True
  ✅ T2 七子系统 7/7 + Table1 最小/最大实现
     （mini-swe 线性 while / OpenHands 事件溯源均原句命中）
  ✅ T3 双缺席1：无 agentic 框架（含 Gemini CLI 不用自家框架）
  ✅ T4 双缺席2：无向量代码检索（ripgrep/tree-sitter/glob）
  ✅ T5 SKILL.md 9/11 领先 MCP 8/11；ACP 6 系统
  ✅ T6 Mini-SWE-Agent ~100 行七子系统存在证明
  ✅ T7 不 benchmark 不排名；13 观察 + 29 模式
排查实录：T2 首跑 6/7——"Memory & context"被 HTML 实体 & 挡住
→ 实体解码修正后 7/7（过程保留）。
```

```
cd exercise
python verify_claims.py    # 7 断言，exit=0
```

**诚实声明：**核对为原句级文本断言（论文全文已存档可复查），未复算 400 万行语料统计本身（需 clone 11 个仓库逐行验证，超出本机范围）；帖内宣称与论文原文一致性 100%；SWE-bench 成绩为论文转述的自报数字（论文自己加了方法论警示）。

**学习来源**：@程序员鱼皮 推文（https://x.com/yupi996/status/2106760754997203304）；论文 arXiv:2609.00006

（CC BY 4.0，全文已存 exercise/paper.html）。

**产物**：本指南 + exercise/verify_claims.py（7/7）+ exercise/paper.html + 实验日志 + 知识提案。生成：2026-10-09，主控 worker kimi-pool-20261007-w1。